Practical No. : 4

Title: Building a Web Crawler to Pull Product Information and Links from an E-commerce Website

Aim: The objective of this project is to develop a web crawler that can extract product details such as product name, price, and links from an e-commerce website. This data will be saved in a structured format like CSV for further use in data analysis or application development.

Prerequisite:o successfully build and run the web crawler, the following tools, libraries, and concepts are required:

1. **Basic Python Programming**: Understanding Python syntax, loops, and functions.
2. **Web Scraping Libraries**: Familiarity with libraries like `requests` and `BeautifulSoup`.
3. **HTML and CSS Knowledge**: Ability to inspect web page structures (HTML tags, classes, etc.) for identifying the relevant data.
4. **Pandas**: Used for saving the extracted data into a CSV format.
5. **Optional: Selenium**: Needed if the webpage is rendered dynamically by JavaScript.
6. **Web Browser Developer Tools**: To inspect and identify the correct HTML tags for scraping.
7. **ChromeDriver**: Needed for running Selenium to handle dynamic content if required.

Objectives:

● The objective of this is to develop a web crawler that efficiently extracts relevant product information, such as product names, prices, and links, from an e-commerce website. The extracted data will be organized and saved in a CSV file for further analysis or integration into other applications, such as price comparison tools or inventory management systems. This project aims to demonstrate the use of Python for web scraping and handling both static and dynamic web content.

Input:

URL of the E-commerce Product Page: This is the URL where the web crawler will extract product data such as product names, prices, and links. Example URL: `http://localhost:8000/product.html"`

Outcome:

● **CSV File**: The extracted product data will be saved in a `CSV` file, containing product names, prices, and links to the product pages.

Theory:

A **web crawler** is a program that automatically fetches and extracts information from web pages. In this project, the web crawler will be built using Python and web scraping tools like `requests` and `BeautifulSoup`.

- **Web Scraping**:
  This involves retrieving the HTML content of a webpage and extracting specific data from it using a process called parsing.
  - **requests** library is used to send a GET request to the server and download the webpage's HTML content.

    - **BeautifulSoup** is used to parse the HTML content and extract the desired data like product names, prices, and links.
    - **Selenium** is used if the website uses JavaScript to dynamically render the content after the initial page load.
- **HTML Structure**:
  The HTML structure of e-commerce product pages is generally uniform, with product names, prices, and links stored in identifiable tags and class attributes such as `<h2>` for product names, `<span>` for prices, and `<a>` for links.

Web Scraping Steps:

1. **Send HTTP Request**:
   Send an HTTP GET request using `requests` to retrieve the HTML content of the target webpage.
2. **Parse HTML**:
   Use `BeautifulSoup` to parse the retrieved HTML document. This allows easy traversal and extraction of data from specific HTML tags or classes.
3. **Extract Product Information**:
   Use `find_all()` to locate all product containers (e.g., `<div>` tags with a specific class name). Within each container, extract the product name, price, and link by navigating the HTML structure.
4. **Handle JavaScript-Rendered Content (Optional)**:
   If the content is dynamically loaded via JavaScript, use **Selenium** to fully render the page and retrieve the final HTML before parsing with `BeautifulSoup`.
5. **Save Data to CSV**:
   Once the information is extracted, use `pandas` to save the data in a CSV format for further use.

Conclusion :

By using this web crawler, we were able to successfully extract product details like product name, price, and link from an e-commerce website. The data was parsed from the webpage's HTML structure and saved in a structured CSV format. This approach demonstrates how web scraping techniques can be used for various applications such as price monitoring, competitive analysis, and product cataloging.